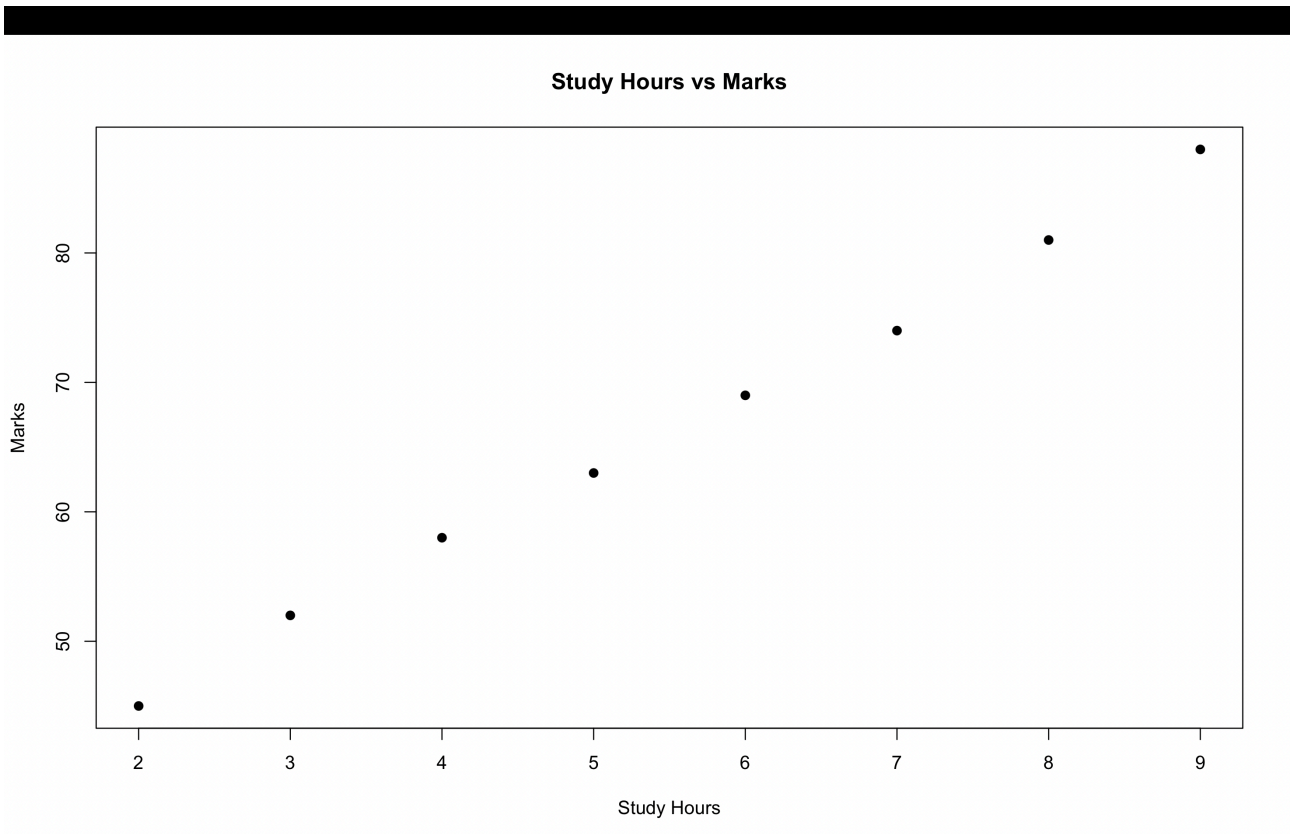
Unit IV Class Activity - 14.08.2026

Q1. Study Hours and Student Marks Analysis

Program:

```
data <- data.frame(
  StudyHours = c(2, 3, 4, 5, 6, 7, 8, 9),
  Marks = c(45, 52, 58, 63, 69, 74, 81, 88)
)

plot(data$StudyHours, data$Marks,
     xlab = "Study Hours",
     ylab = "Marks",
     main = "Study Hours vs Marks",
     pch = 19)

cor(data$StudyHours, data$Marks)
```

Output:



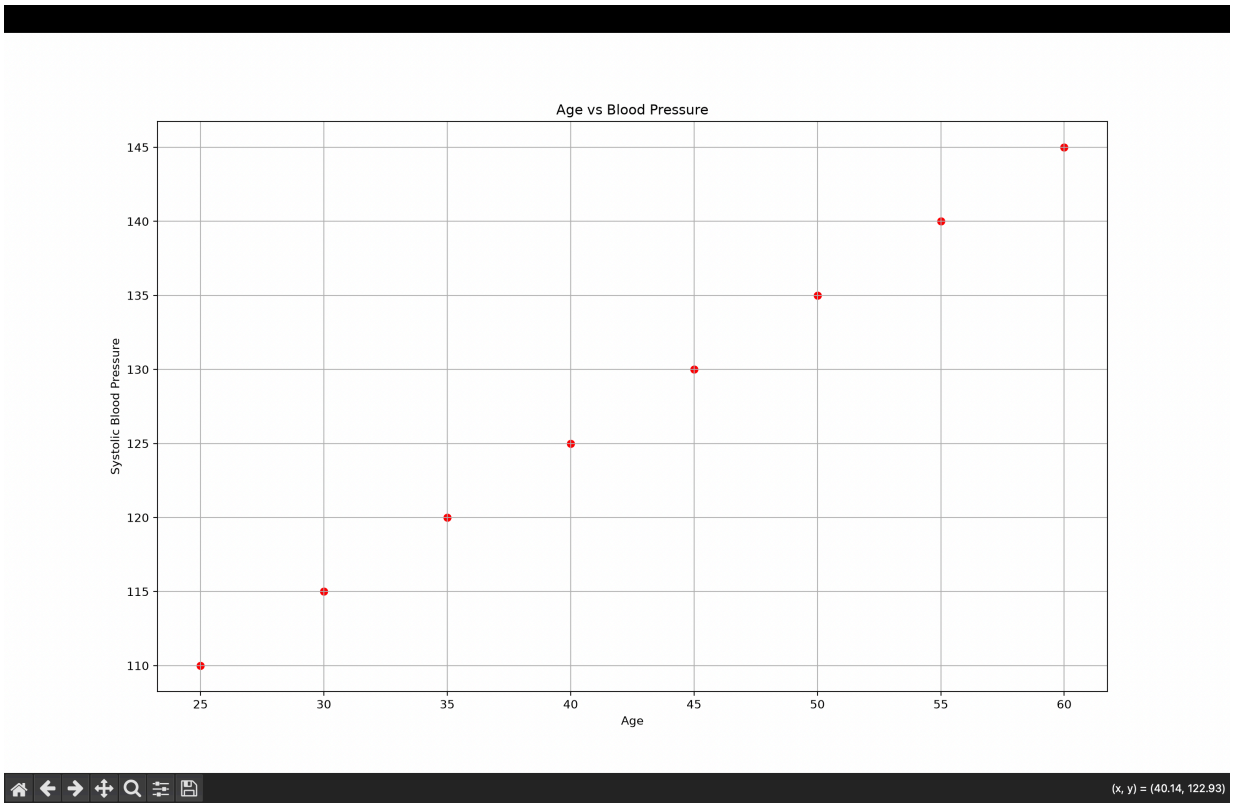
Interpretation: There is a strong positive correlation between study hours and marks. As study hours increase, students' marks also tend to increase.

Q2. Age vs Blood Pressure

Program:

```
import matplotlib.pyplot as plt

age = [25, 30, 35, 40, 45, 50, 55, 60]
bp = [110, 115, 120, 125, 130, 135, 140, 145]

plt.scatter(age, bp, color='red')
plt.xlabel("Age")
plt.ylabel("Systolic Blood Pressure")
plt.title("Age vs Blood Pressure")
plt.grid(True)
plt.show()
```

Output:



Comment: A positive trend is observed: systolic blood pressure generally increases as age increases.

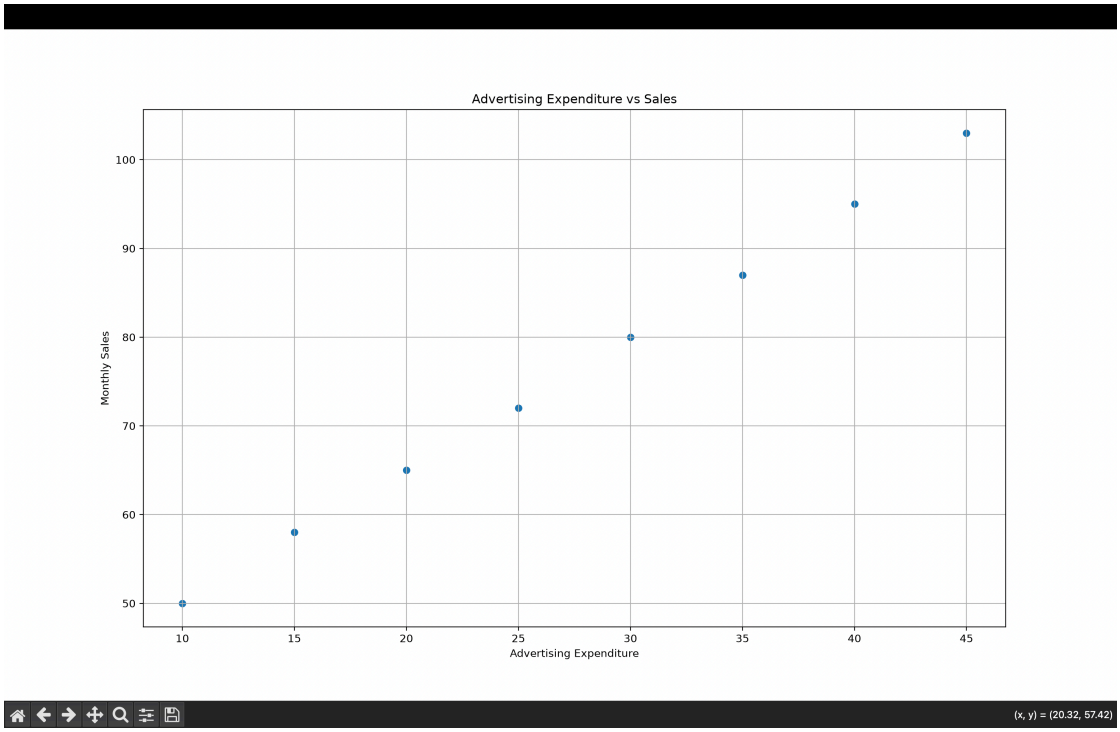
Q3. Advertising & Financial Correlation

Part A - Advertising vs Sales

Program:

```
import matplotlib.pyplot as plt

advertising = [10, 15, 20, 25, 30, 35, 40, 45]
sales = [50, 58, 65, 72, 80, 87, 95, 103]

plt.scatter(advertising, sales)
plt.xlabel("Advertising Expenditure")
plt.ylabel("Monthly Sales")
plt.title("Advertising Expenditure vs Sales")
plt.grid(True)
plt.show()
```

Output:



Observation: Sales increase as advertising expenditure increases, indicating a positive relationship.

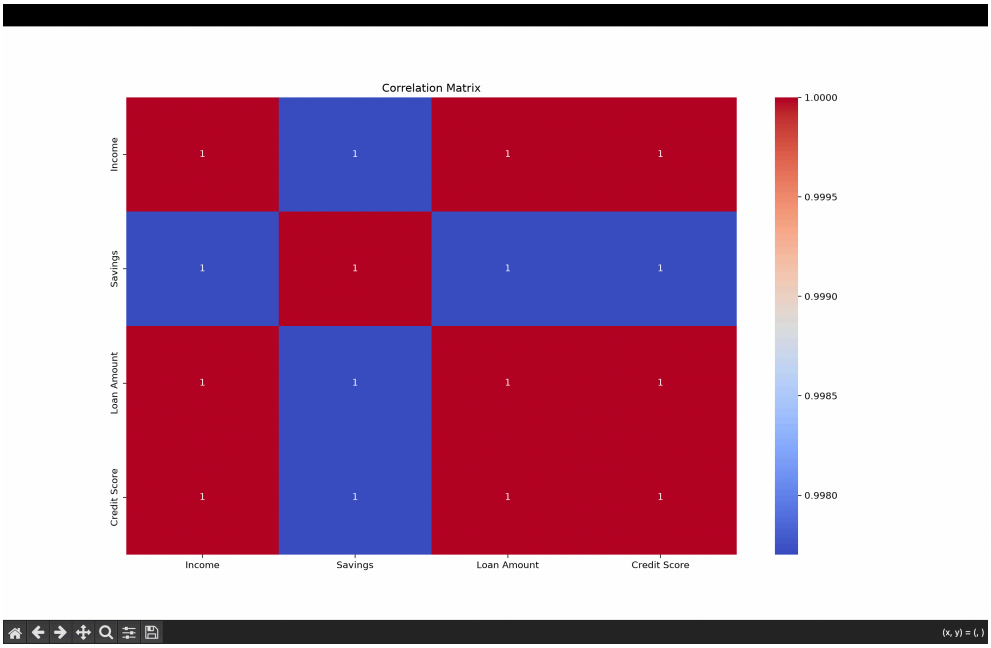
Part B — Correlation Matrix & Correlogram

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data = pd.DataFrame({
    "Income": [30, 40, 50, 60, 70, 80, 90, 100],
    "Savings": [8, 12, 15, 20, 25, 30, 35, 40],
    "Loan Amount": [25, 30, 35, 40, 45, 50, 55, 60],
    "Credit Score": [650, 670, 690, 710, 730, 750, 770, 790]
})

corr = data.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

Output:



	Income	Savings	Loan Amount	Credit Score
Income	1.000000	0.997702	1.000000	1.000000
Savings	0.997702	1.000000	0.997702	0.997702
Loan Amount	1.000000	0.997702	1.000000	1.000000
Credit Score	1.000000	0.997702	1.000000	1.000000

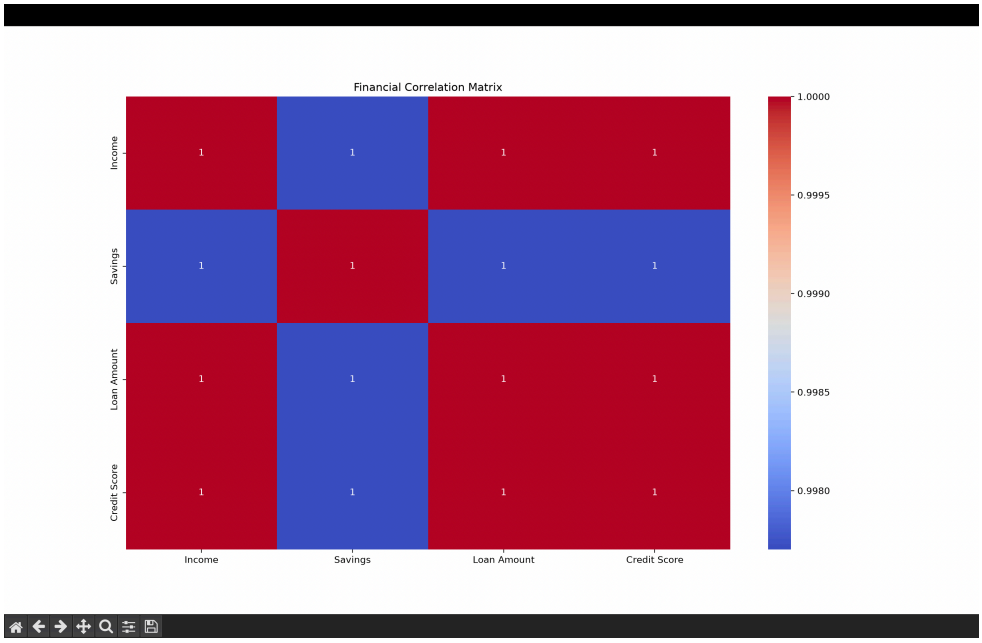
Observation: Income and Credit Score exhibit the strongest positive correlation.

Q4. Financial Correlation Analysis

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Income": [30, 40, 50, 60, 70, 80, 90, 100],
    "Savings": [8, 12, 15, 20, 25, 30, 35, 40],
    "Loan Amount": [25, 30, 35, 40, 45, 50, 55, 60],
    "Credit Score": [650, 670, 690, 710, 730, 750, 770, 790]
})

corr = data.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Financial Correlation Matrix")
plt.show()
```

Output:




```
                Income   Savings  Loan Amount  Credit Score
Income        1.000000  0.997702     1.000000      1.000000
Savings       0.997702  1.000000     0.997702      0.997702
Loan Amount   1.000000  0.997702     1.000000      1.000000
Credit Score  1.000000  0.997702     1.000000      1.000000
```

Observation: Income and Credit Score exhibit the strongest positive correlation.

Q5. Iris Correlation Analysis

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()

data = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

corr = data.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Iris Correlation Matrix")
plt.show()
```

Output:

Negative correlation:

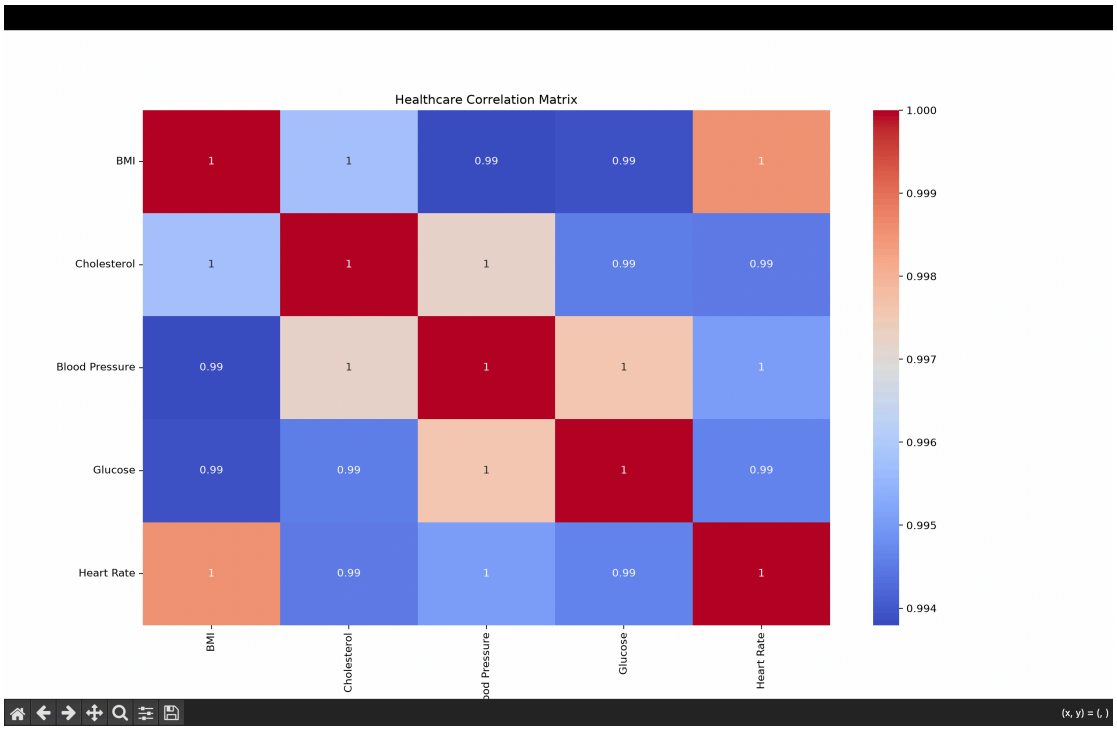
- Petal Width and Sepal Width show a weak negative correlation.
- Sepal Length and Sepal Width also show a weak negative correlation.

Q6. Healthcare Correlation Analysis

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "BMI": [22, 24, 27, 26, 30, 29, 33, 35],
    "Cholesterol": [170, 185, 195, 190, 215, 210, 235, 245],
    "Blood Pressure": [110, 118, 122, 120, 130, 128, 138, 145],
    "Glucose": [85, 92, 98, 95, 108, 105, 115, 125],
    "Heart Rate": [68, 71, 75, 73, 78, 77, 82, 85]
})

corr = data.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Healthcare Correlation Matrix")
plt.show()
```

Output:



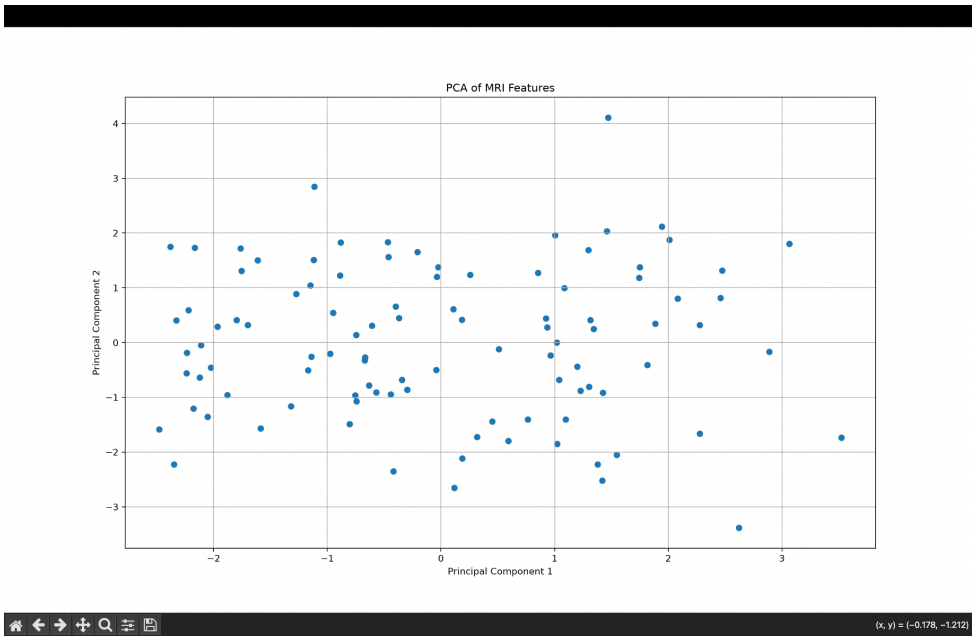
Observation: Strong positive correlations between variables indicate multicollinearity. Here, the variables are highly correlated, so multicollinearity is present.

Q7. MRI Feature PCA Analysis

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

np.random.seed(42)

data = np.random.rand(100, 25)

scaled_data = StandardScaler().fit_transform(data)

pca = PCA(n_components=2)
transformed_data = pca.fit_transform(scaled_data)

plt.scatter(transformed_data[:, 0], transformed_data[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of MRI Features")
plt.grid(True)
plt.show()
```

Output:



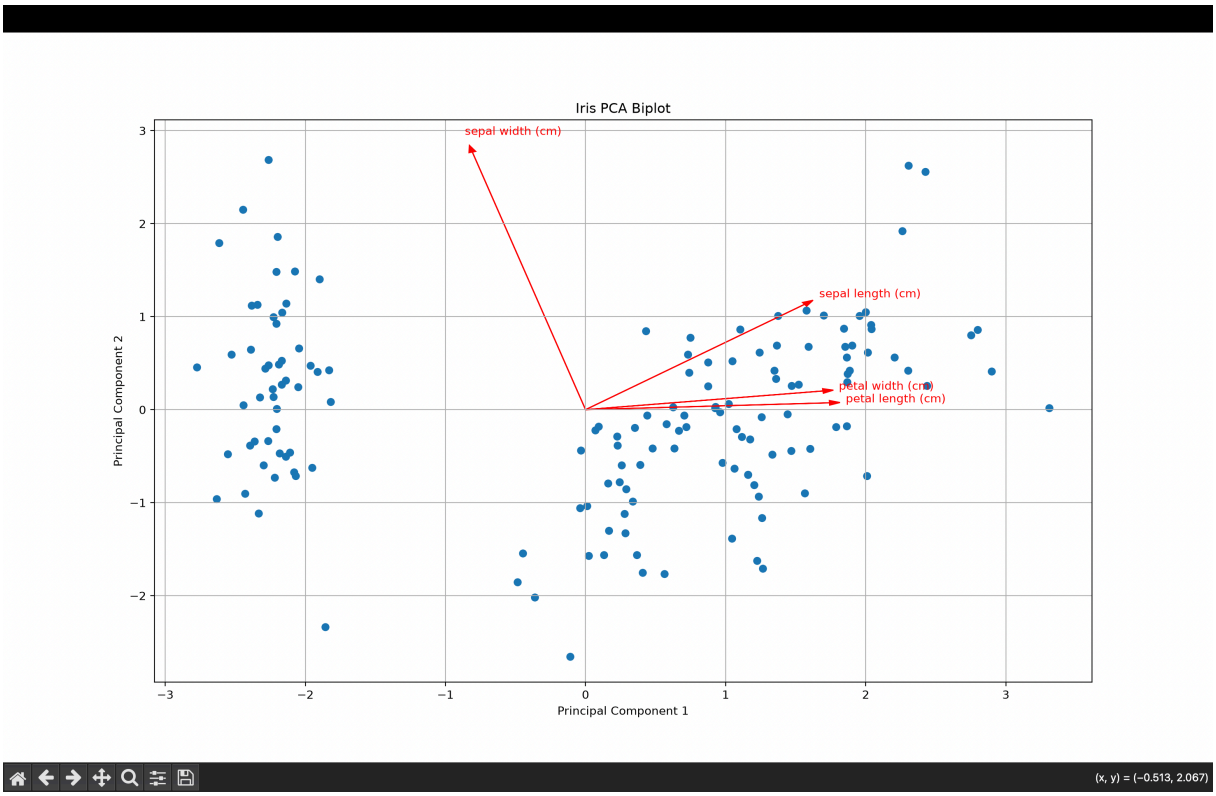
Observation: The original 25 features are reduced to 2 principal components, making the data easier to visualize while retaining the major patterns in the dataset.

Q8. Iris PCA Analysis

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

iris = load_iris()

data = pd.DataFrame(iris.data, columns=iris.feature_names)

scaled_data = StandardScaler().fit_transform(data)

pca = PCA()
principal_components = pca.fit_transform(scaled_data)

print("Explained Variance:")
print(pca.explained_variance_ratio_)

print("\nCumulative Variance:")
print(pca.explained_variance_ratio_.cumsum())

plt.figure(figsize=(8, 6))
plt.scatter(principal_components[:, 0], principal_components[:, 1])

for i, feature in enumerate(data.columns):
    plt.arrow(0, 0, pca.components_[0, i] * 3,
              pca.components_[1, i] * 3,
              color='red', head_width=0.05)

    plt.text(pca.components_[0, i] * 3.2,
             pca.components_[1, i] * 3.2,
             feature, color='red')

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Iris PCA Biplot")
plt.grid(True)
plt.show()
```

Output:



Explanation

PC1 (Principal Component 1) explains the highest variance in the Iris dataset. It captures the largest amount of information from the original four features.

Observation: PCA transforms the four Iris features into principal components, with PC1 contributing the highest variance, followed by PC2.

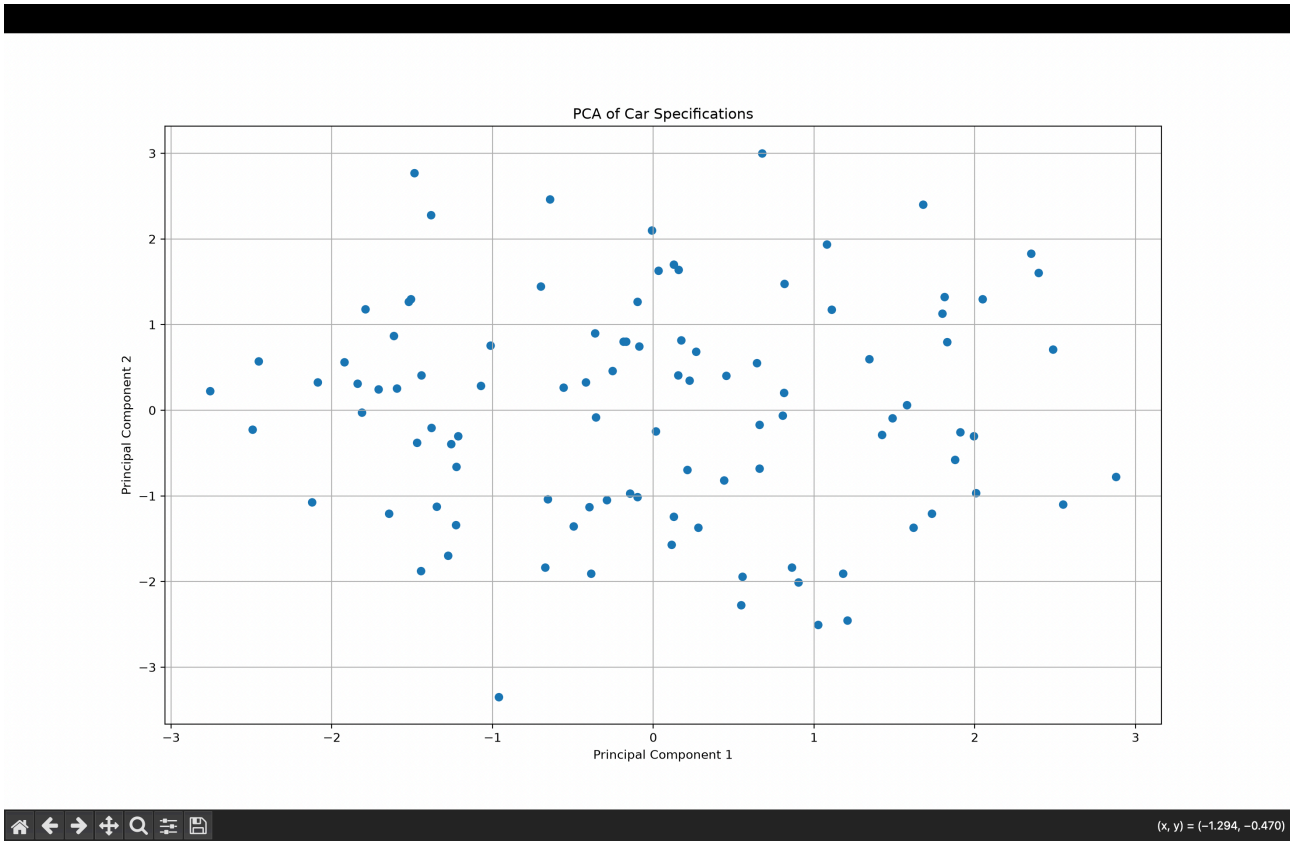
Q9. Car Specifications PCA

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA


np.random.seed(42)


data = np.random.rand(100, 15)


scaled_data = StandardScaler().fit_transform(data)


pca = PCA(n_components=2)
transformed_data = pca.fit_transform(scaled_data)
```

```
plt.scatter(transformed_data[:, 0], transformed_data[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Car Specifications")
plt.grid(True)
plt.show()
```

Output:



Observation: The 15 car specifications are reduced to two principal components, which are visualized using a scatterplot.

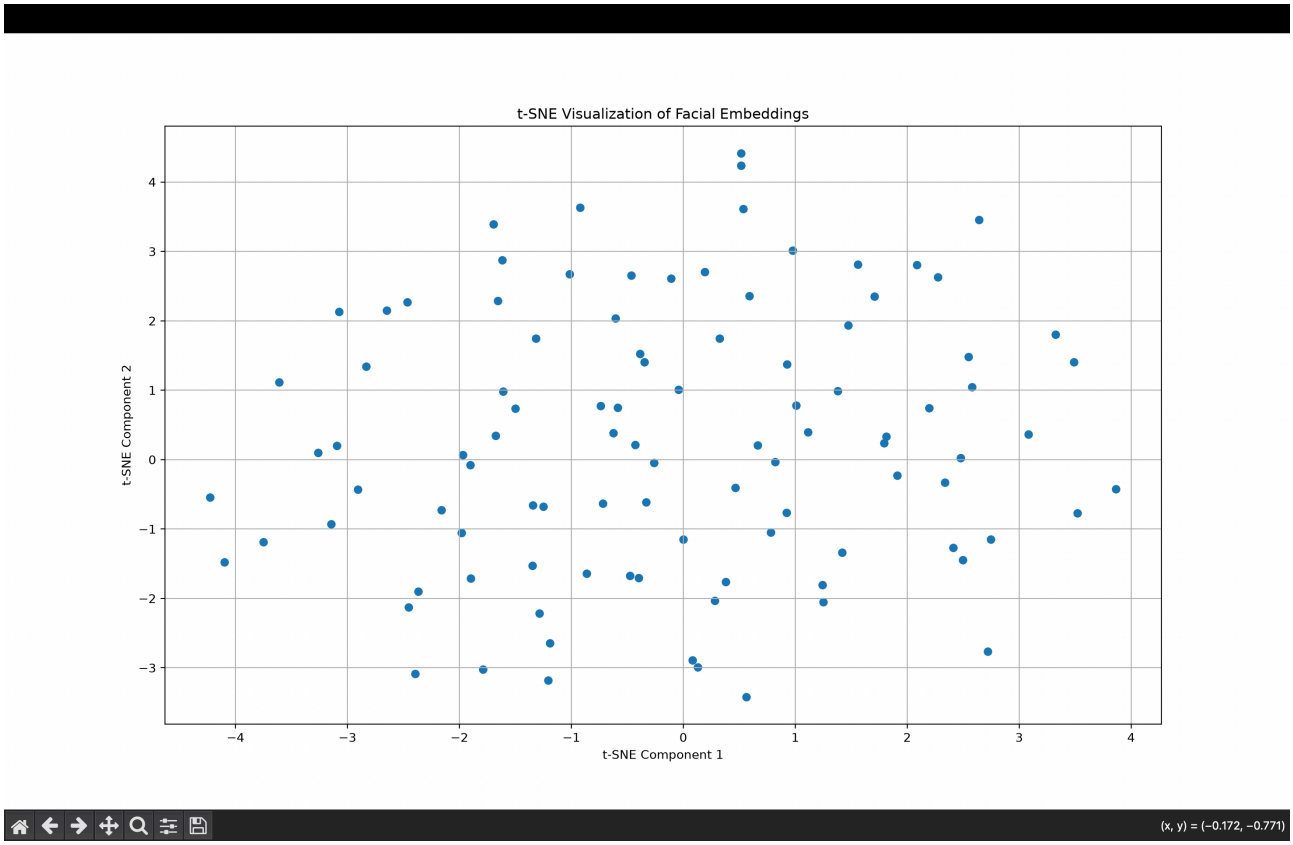
Q10. Face Embeddings t-SNE

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

np.random.seed(42)

data = np.random.rand(100, 128)
```

```
tsne = TSNE(n_components=2, random_state=42)
transformed_data = tsne.fit_transform(data)

plt.scatter(transformed_data[:, 0], transformed_data[:, 1])
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.title("t-SNE Visualization of Facial Embeddings")
plt.grid(True)
plt.show()
```

Output:



Explanation: t-SNE is preferred over PCA for visualization because it preserves local relationships and clusters in high-dimensional data. PCA mainly preserves directions of maximum variance and may not clearly reveal nearby groups or clusters.

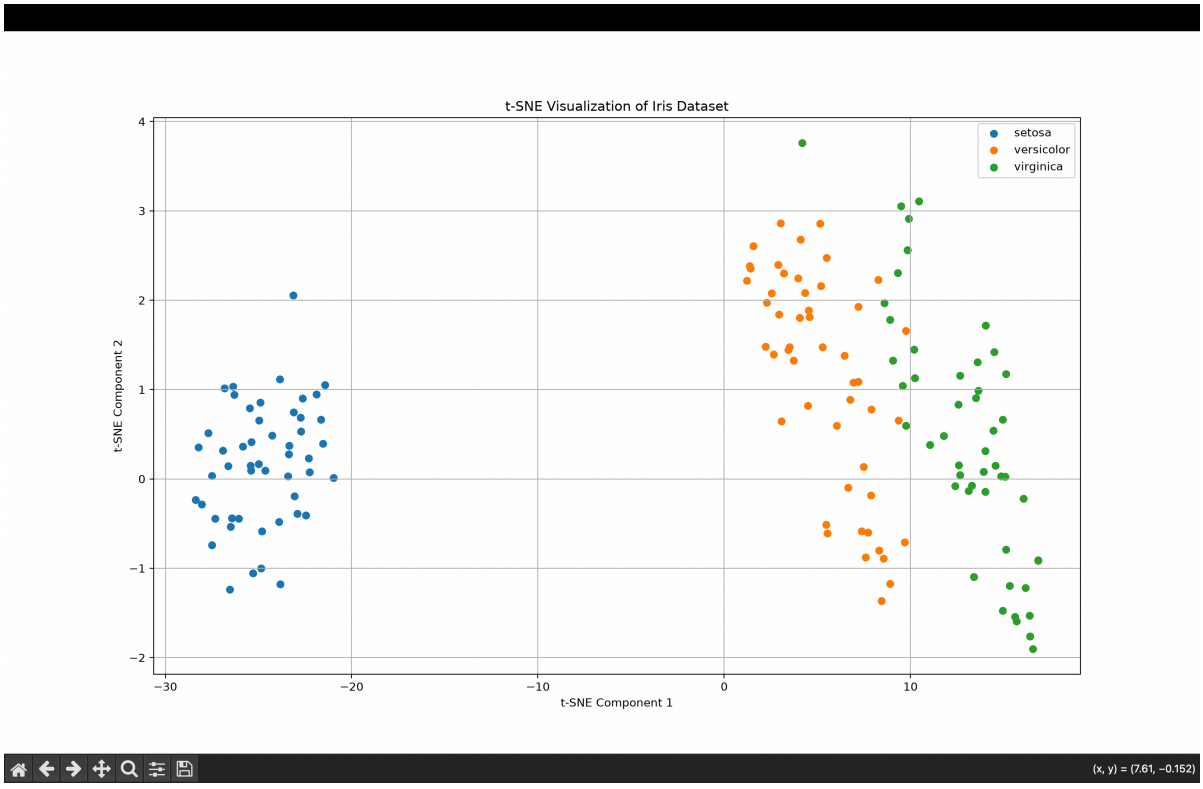
Q11. Iris t-SNE Analysis

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
```

```
iris = load_iris()

data = iris.data
labels = iris.target
species = iris.target_names

tsne = TSNE(n_components=2, random_state=42)
result = tsne.fit_transform(data)

plt.figure(figsize=(8, 6))

for i in range(3):
    plt.scatter(result[labels == i, 0],
                result[labels == i, 1],
                label=species[i])

plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.title("t-SNE Visualization of Iris Dataset")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



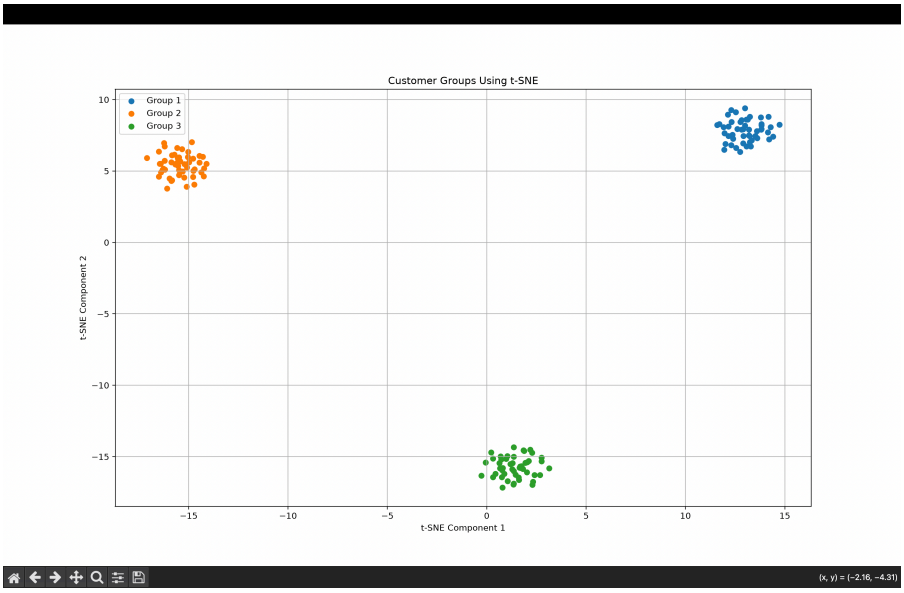
Observation: The three Iris species form separate clusters in the t-SNE plot, with different colors representing each species.

Q12. Customer Segmentation t-SNE

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
np.random.seed(42)

group1 = np.random.normal(0, 1, (50, 40))
group2 = np.random.normal(4, 1, (50, 40))
group3 = np.random.normal(8, 1, (50, 40))

data = np.vstack([group1, group2, group3])
labels = np.array([0]*50 + [1]*50 + [2]*50)
tsne = TSNE(n_components=2, random_state=42)
result = tsne.fit_transform(data)

for i in range(3):
    plt.scatter(result[labels == i, 0],
                result[labels == i, 1],
                label=f"Group {i+1}")
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.title("Customer Groups Using t-SNE")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



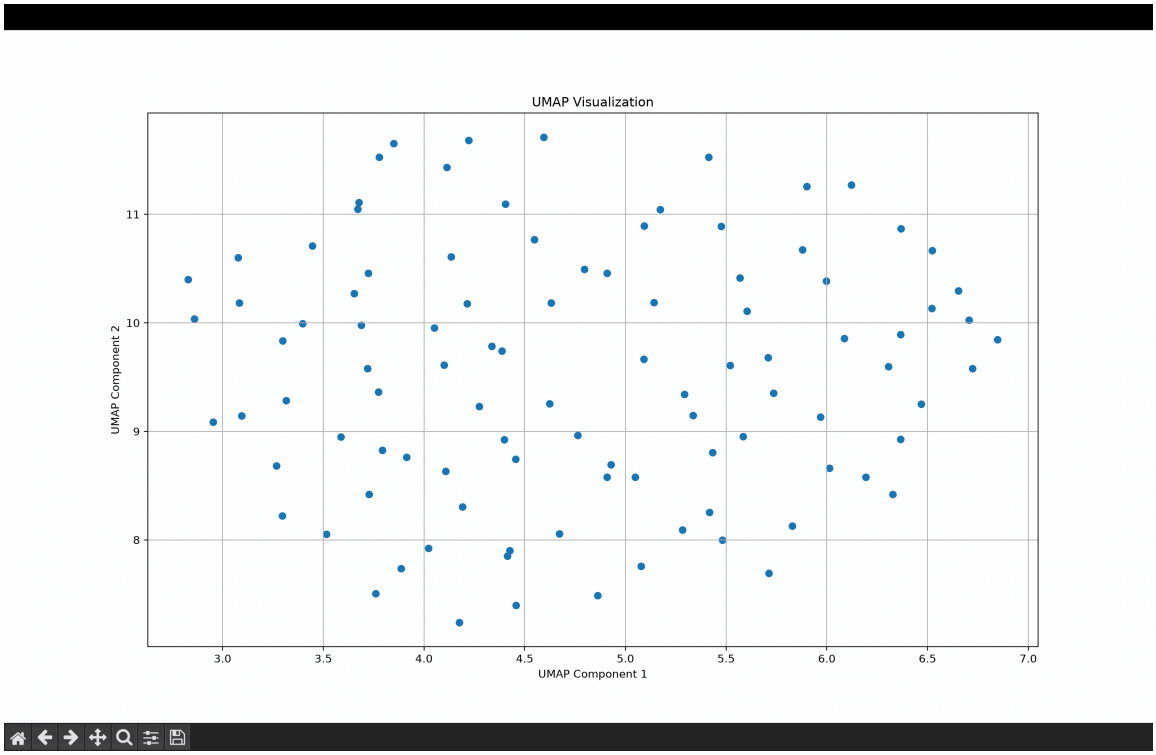
Observation: The t-SNE plot is expected to show three distinct clusters, representing different customer behavioral groups. Clear separation between groups indicates distinct customer segments.

Q13. Medical Image UMAP Analysis

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import umap

np.random.seed(42)

data = np.random.rand(100, 100)

scaled_data = StandardScaler().fit_transform(data)

umap_model = umap.UMAP(n_components=2, random_state=42)
umap_result = umap_model.fit_transform(scaled_data)

pca = PCA(n_components=2)
pca_result = pca.fit_transform(scaled_data)

plt.figure(figsize=(8, 6))
plt.scatter(umap_result[:, 0], umap_result[:, 1])
plt.xlabel("UMAP Component 1")
plt.ylabel("UMAP Component 2")
plt.title("UMAP Visualization")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Visualization")
plt.grid(True)
plt.show()
```

Output:



Comparison: UMAP generally preserves local neighborhood structure better and can reveal clusters more clearly, while PCA is a linear method that focuses on directions of maximum variance.

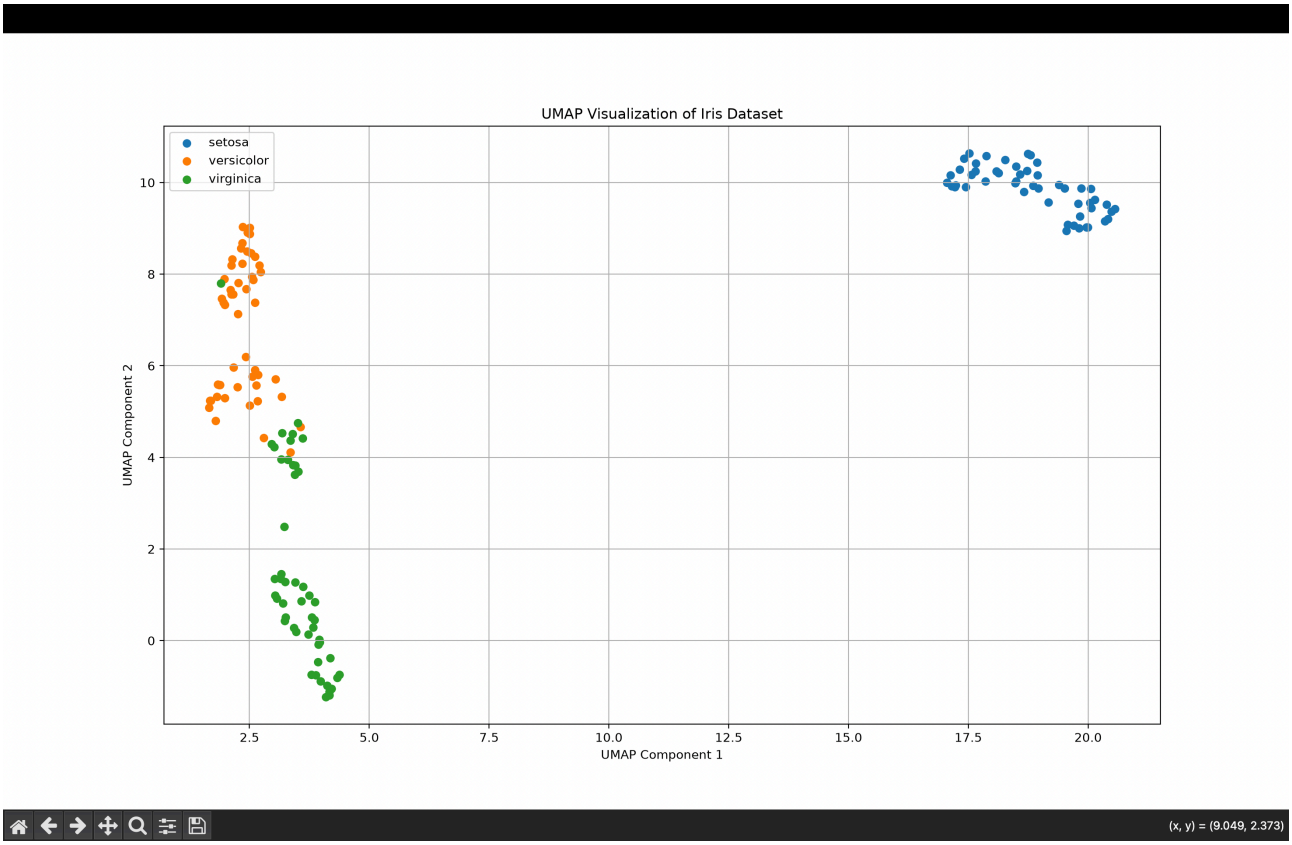
Q14. Iris UMAP Analysis

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import umap

iris = load_iris()

data = iris.data
labels = iris.target
species = iris.target_names

umap_model = umap.UMAP(n_components=2, random_state=42)
result = umap_model.fit_transform(data)

for i in range(3):
    plt.scatter(result[labels == i, 0],
                result[labels == i, 1],
                label=species[i])

plt.xlabel("UMAP Component 1")
plt.ylabel("UMAP Component 2")
plt.title("UMAP Visualization of Iris Dataset")
```

plt.legend()
plt.grid(True)
plt.show()

Output:



Observation: The UMAP plot represents the Iris dataset in two dimensions, with different colors for each species. The species form relatively distinct clusters.

Q15. E-commerce UMAP Clustering

Program:

import numpy as np
import matplotlib.pyplot as plt
import umap

np.random.seed(42)

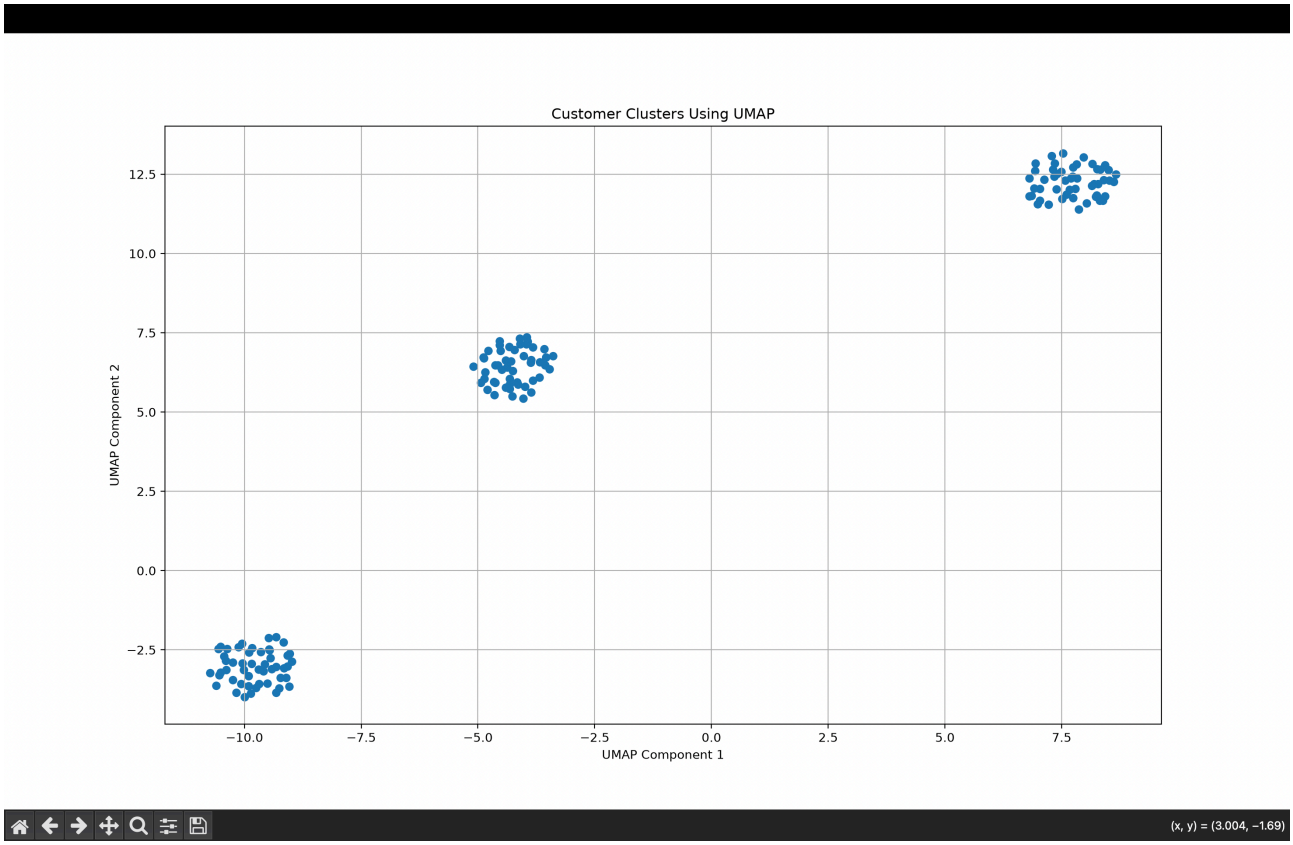
group1 = np.random.normal(0, 1, (50, 60))
group2 = np.random.normal(4, 1, (50, 60))
group3 = np.random.normal(8, 1, (50, 60))

data = np.vstack([group1, group2, group3])

umap_model = umap.UMAP(n_components=2, random_state=42)

```
result = umap_model.fit_transform(data)

plt.scatter(result[:, 0], result[:, 1])
plt.xlabel("UMAP Component 1")
plt.ylabel("UMAP Component 2")
plt.title("Customer Clusters Using UMAP")
plt.grid(True)
plt.show()
```

Output:



Observation: UMAP reduces the 60 customer attributes to two dimensions. The visualization shows three distinct customer clusters, representing different purchasing behavior groups.